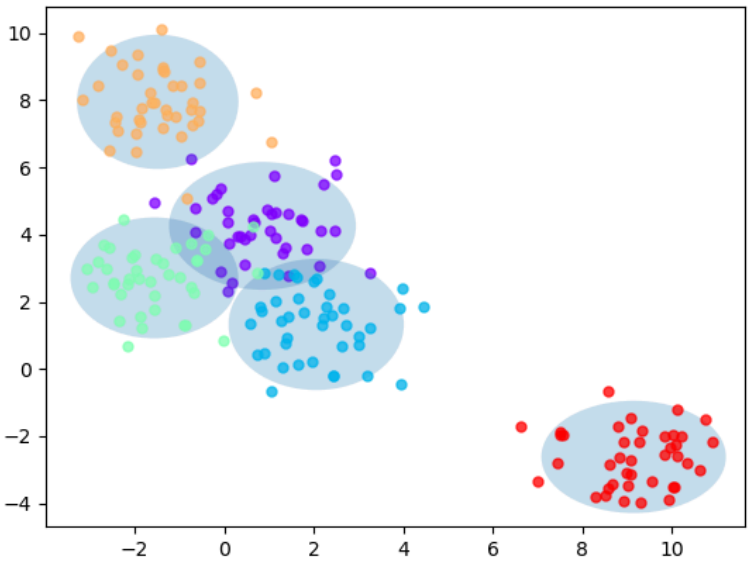
Lab 3

Assignment 1. *Write a function,* `mlParams(X,labels)`*, that computes the MLestimates of* $\boldsymbol{\mu}_k$ *and* $\boldsymbol{\Sigma}_k$ *for the different classes in the dataset.* `X` *here is a set of row vectors, and* `labels` *are the class labels for each of the data points (again, ignore the* `W` *argument for now). The function should return a C × d-array* `mu` *that contains the class means, a C × d × d-array* `sigma` *that contains the class covariances. The covariance should be implemented using your own code and not by applying a library function.*

Use the provided function, `genBlobs()`*, that returns Gaussian distributed data points together with class labels, to generate some test data. Compute the ML-estimates for the data and plot the 95%-confidence interval using the function* `plotGaussians`*.*



Assignment 2.
(1) Write a function `computePrior(labels)` *that estimates and returns the class prior in* `X` *(ignore the* `W` *argument).*
(2) Write a function `classifyBayes(X,prior,mu,sigma)` *that computes the discriminant function values for all classes and data points, and classifies each point to belong to the max discriminant value. The function should return a length N vector containing the predicted class value for each point.*

No text answer needed

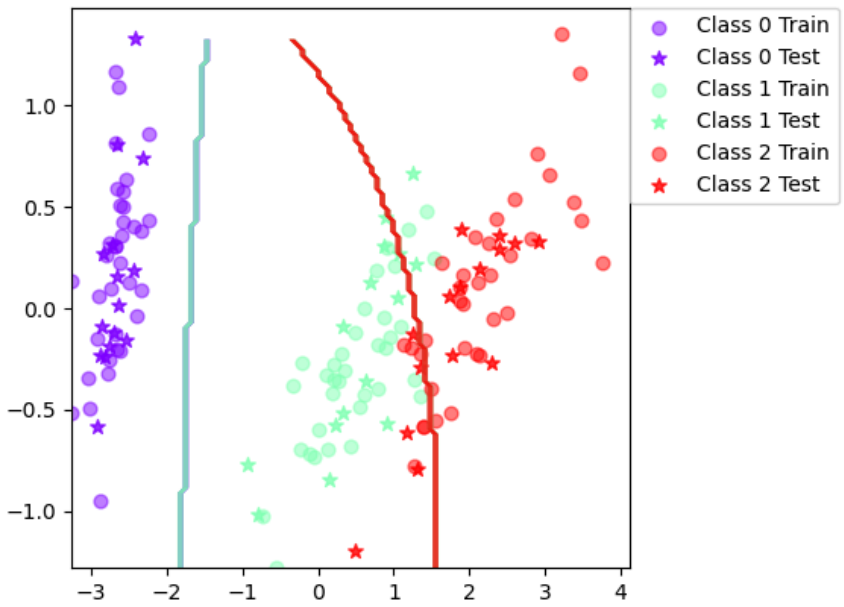
Assignment 3. *We now have all the functions we need for doing the training and classification. Use the provided function* `testClassifier` *to test the accuracy for the vowels and iris datasets.* `testClassifier` *runs a loop that does the following things:*

0. Uses the provided random partitioning function to split the dataset into a training and test dataset.
1. Train your classifier on the training partition.
2. Evaluate the performance of the classifier on the test partition.

Run `testClassifier` *for the datasets and take note of the accuracies. Use* `plotBoundary` *to plot the decision boundary of the 2D iris dataset.*

Answer the following questions:
(1) When can a feature independence assumption be reasonable and when not?
(2) How does the decision boundary look for the Iris dataset? How could one improve the classification results for this scenario by changing classifier or, alternatively, manipulating the data?

Iris (Bayes)		Vowel (Bayes)	
Trial	**Accuracy**	**Trial**	**Accuracy**
0	84.4	0	61
10	95.6	10	66.2
20	93.3	20	74
30	86.7	30	66.9
40	88.9	40	59.7
50	91.1	50	64.3
60	86.7	60	66.9
70	91.1	70	63.6
80	86.7	80	62.3
90	91.1	90	70.8
Final mean classification accuracy: **89**		Final mean classification accuracy: **64.7**	
Standard deviation: **4.16**		Standard deviation: **4.03**	

Iris	

1.When can a feature independence assumption be reasonable and when not?
The feature independence assumption simplifies the computation and makes the algorithm more efficient (the joint probability of features can be computed as the product of individual feature probabilities). This effect is most noticeable in high-dimensional feature spaces. Can still capture essential relationships between features and classes.

Bad in scenarios where feature dependencies significantly affect the classification decision.

The assumption breaks down when features are strongly correlated or interact in complex ways. For example, in image processing, neighboring pixels are usually highly correlated because they tend to have similar values. Assuming independence between pixels would be unrealistic, and models that account for these correlations, such as convolutional neural networks (CNNs), are more appropriate. In this case of image processing, models that account for feature dependencies are necessary to accurately capture the relationships between variables. Covariance value should be added in the sigma matrix.

2.How does the decision boundary look for the Iris dataset? How could one improve the classification results for this scenario by changing classifier or, alternatively, manipulating the data?

In the figure we can see two parts of the decision boundary that divides the three classes. The first part is quite straight and effectively separates class 0 from class 1 (since class 0 has more distinct features compared to the other classes). The other decision boundary part is curved and doesn't completely separate class 1 and class 2. It looks like class 1 and class 2 almost can be separated by a straight line, so a support vector machine (SVM) with a

linear kernel could eventually improve the classification results. It would probably be a good idea to have some slack too since the data points are “overlapping” a little bit.

Assignment 4. *Extend the old mlParams function to* `mlParams(X, labels, W)` *that handles weighted instances. Again* `X` *is an N × d matrix of feature vectors, labels a length N vector containing the corresponding labels and* `W` *is a N × 1 matrix of weights.*

No text answer needed

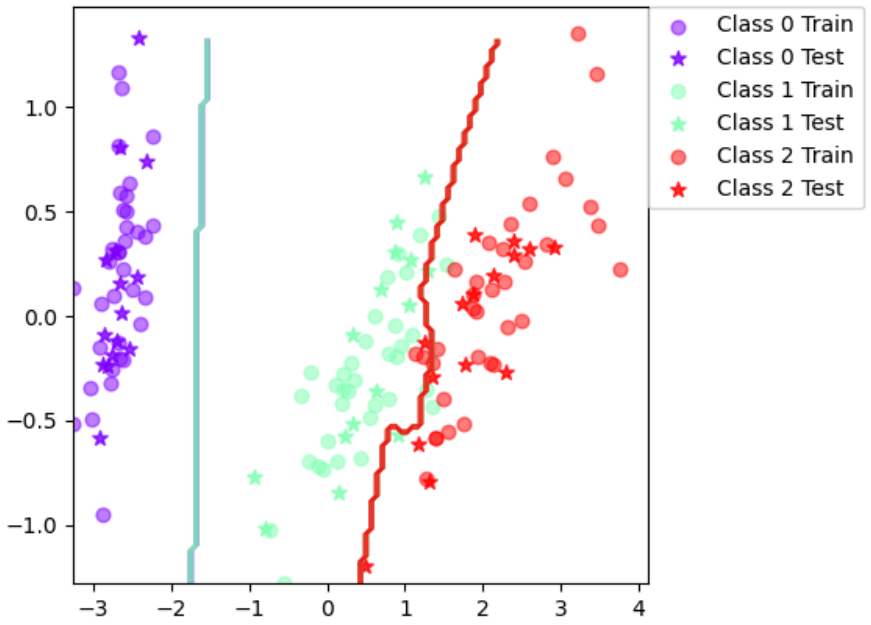
Assignment 5.
(1) Modify `computePrior` *to have the signature* `computePrior(labels, W)`*, taking the boosting weights* $\boldsymbol{\omega}$ *into account. We can look at the weights as taking a particular training point* $\boldsymbol{x}_i$ *into account* ω_i *times. So if previously there were* N_k *points in a particular class, we should now think about “how many times we count” each point. Note that the prior probabilities should still sum to one.*
(2) Implement the Adaboost algorithm and apply it to the Bayes classifier. Design a function `trainBoost(base classifier, X, labels, T)` *that generates a set of boosted hypotheses, where the parameter T determines the number of hypotheses. Use the modified* `computePrior(labels, W)`*. Note that a new classifier of type* `base_classifier` *can be trained by calling*
`new_classifier = base_classifier.trainClassifier(X, y, W)`,
which can then be used for classification by calling `new_classifier.classify(X)`.
(3) Design a function `classifyBoost(X, classifiers, alphas, Nclasses)` *that classifies the instances in data by means of the aggregated boosted classifier according to Equation 15. The resulting classifications are returned in the vector* `yPred`*.*

Observe: *The return parameter* `alphas`*, a length T list, holds the classifier vote weights* α^t*. classifiers should be a length T list of trained classifiers.* `yPred` *will be a length N vector of predicted class labels. Note that you have to compute and store all the hypotheses generated with* `classifiers[t].classify(data, labels, W)` *for each of the different distributions* ω^{t+1} *and later aggregate their classifications to obtain the overall classification.*

Compute the classification accuracy of the boosted classifier on some data sets using `testClassifier` *from* `labfuns.py` *and compare it with those of the basic classifier on the vowels and iris data sets (see Assignment 3):*
(1) Is there any improvement in classification accuracy? Why/why not?
(2) Plot the decision boundary of the boosted classifier on iris and compare it with that of the basic. What differences do you notice? Is the boundary of the boosted version more complex?
(3) Can we make up for not using a more advanced model in the basic classifier (e.g. independent features) by using boosting?

You may use the function `plotBoundary` *provided in* `labfuns.py` *to plot the decision boundary for different datasets and parameters.*

Iris (Bayes, Boosted)			Vowel (Bayes, Boosted)		
Trial	**Accuracy**	**Difference**	**Trial**	**Accuracy**	**Difference**
0	95.6	11.2	0	76.6	15.6
10	100	4.4	10	86.4	20.2
20	93.3	0	20	83.1	9.1
30	91.1	4.4	30	80.5	13.6
40	97.8	8.9	40	72.7	13
50	93.3	2.2	50	76	11.7
60	93.3	6.6	60	81.8	14.9
70	97.8	6.7	70	82.5	18.9
80	95.6	8.9	80	79.9	17.6
90	93.3	2.2	90	83.1	12.3
Final mean classification accuracy: **94.7**			Final mean classification accuracy: **80.2**		
Difference: **5.7**			Difference: **15.5**		
Standard deviation: **2.82**			Standard deviation: **3.52**		
Difference: **-1.34**			Difference: **-0.51**		

Iris (Boosted)	

1.Is there any improvement in classification accuracy? Why/why not?
The classification accuracy increased after the boosting for both datasets. Each of the weak classifiers that make up the boosted classifier "vote" for one class for each datapoint. The value of the votes depend on the confidence (α) of the classifier that casts the vote. The class with the highest voting value is then selected. This means less confident classifiers have a smaller impact on the result than more confident ones, but a large group of less confident classifiers can still get a higher combined vote value than a single more confident one.

2.Plot the decision boundary of the boosted classifier on iris and compare it with that of the basic. What differences do you notice? Is the boundary of the boosted version more complex?
The part of the decision boundary that separates class 0 from the other two classes looks the same as it did for the non-boosted classifier. The other part of the decision boundary is however more complex in the boosted version. It is very obvious that this part in the boosted classifier is much more adapted to the data points.

3.Can we make up for not using a more advanced model in the basic classifier (e.g. independent features) by using boosting?
Based on the accuracy results, that seems to be the case. Boosting reduces the bias by each classifier trying to correct the misclassifications made by the previous one. By combining multiple weaker classifiers, a more complex decision boundary can be achieved by boosting.

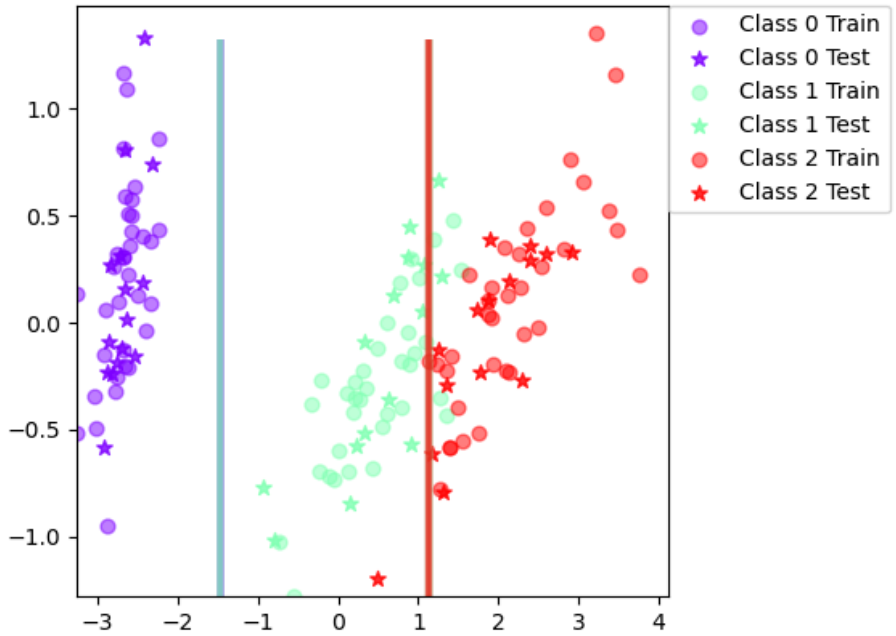
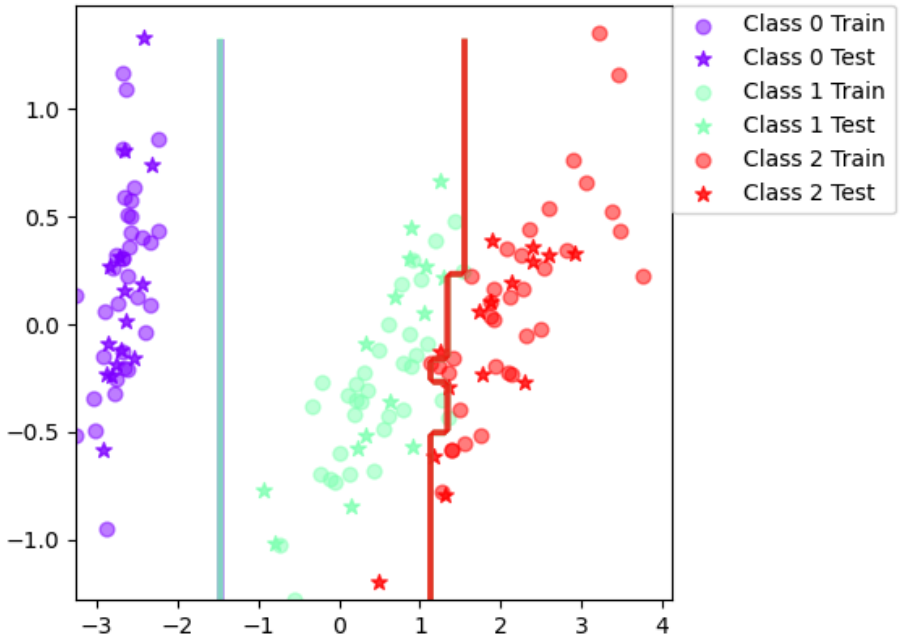
Assignment 6. *We have implemented a class* `DecisionTreeClassifier` *based upon skLearns decision tree classifier. The skLearn implementation is similar to the one used in*

the first lab, however, here the values are continuous and we use the default Gini index to compute the split.

Test the decision tree classifier on the `vowels` *and* `iris` *data sets. Repeat but now by passing it as an argument to the* `BoostClassifier` *object. Answer questions 1-3 in assignment 5 for the decision tree.*

Iris (Decision tree)		**Vowel (Decision tree)**	
Trial	**Accuracy**	**Trial**	**Accuracy**
0	95.6	0	63.6
10	100	10	68.8
20	91.1	20	63.6
30	91.1	30	66.9
40	93.3	40	59.7
50	91.1	50	63
60	88.9	60	59.7
70	88.9	70	68.8
80	93.3	80	59.7
90	88.9	90	68.2
Final mean classification accuracy: **92.4**		Final mean classification accuracy: **64.1**	
Standard deviation: **3.67**		Standard deviation: **4.02**	

Iris (Decision tree, Boosted)			Vowel (Decision tree, Boosted)		
Trial	**Accuracy**	**Difference**	**Trial**	**Accuracy**	**Difference**
0	95.6	0	0	85.7	22.1
10	100	0	10	90.3	21.5
20	95.6	4.5	20	87.7	24.1
30	93.3	2.2	30	93.5	26.6
40	93.3	0	40	83.1	23.4
50	95.6	4.5	50	79.9	16.9
60	88.9	0	60	90.9	31.2
70	93.3	4.4	70	85.7	16.9
80	93.3	0	80	83.1	23.4
90	93.3	4.4	90	85.1	16.9
Final mean classification accuracy: **94.6**			Final mean classification accuracy: **86.6**		
Difference: **2.2**			Difference: **22.5**		
Standard deviation: **3.65**			Standard deviation: **3.21**		
Difference: **-0.02**			Difference: **-0.81**		

Iris (Decision tree)	
Iris (Decision tree, Boosted)	

1.Is there any improvement in classification accuracy? Why/why not?

The classification accuracy increased for both datasets after boosting, but the increase was very small for the Iris dataset which already had a high accuracy before boosting. The effect of the boosting was on the other hand very noticeable for the Vowel dataset. We assume that this is because the Vowel dataset is more complex with more features, so the decision boundary also needs to be more complex.

2.Plot the decision boundary of the boosted classifier on iris and compare it with that of the basic. What differences do you notice? Is the boundary of the boosted version more complex?
The red part of the decision boundary is no longer a straight vertical line but is instead more complex in the boosted version and fits the data points better.

3.Can we make up for not using a more advanced model in the basic classifier (e.g. independent features) by using boosting?
From the decision boundary plots and the classification accuracy result, it seems like boosting can make up for not using a more advanced classifier.

Assignment 7. *If you had to pick a classifier, naive Bayes or a decision tree or the boosted versions of these, which one would you pick? Motivate from the following criteria:*

- *Outliers*
- *Irrelevant inputs: part of the feature space is irrelevant*
- *Predictive power*
- *Mixed types of data: binary, categorical or continuous features, etc.*
- *Scalability: the dimension of the data, D, is large or the number of instances, N, is large, or both.*

<u>Outliers</u>

- NB: Outliers in the training dataset would have an impact on the mean and the variance for Gaussian Naive Bayes.
- DT: Outliers in the training dataset could have a large impact on the decision tree model. However, pruning could probably lessen the impact or the outliers.
- Bo: Outliers can skew the result since each model tries to fix the errors of the previous one. (Outliers could maybe have “heavier weights” since they are more likely to be misclassified at first, making the outliers the focus when boosting)

<u>Irrelevant inputs</u>

- NB: Not sensitive to irrelevant features.
- DT: We assume that decision trees would be quite robust against irrelevant features since the information gain for the splits should be higher for relevant features.
- Bo: We think that the irrelevant features could eventually get less impact when we combine multiple classifiers since the iterative process in boosting focuses on the features that reduce the error. Could probably have some computational impact.

<u>Predictive power</u>

- NB: We believe the Naive Bayes classifier to have high predictive power based on the result we achieved. However, the predictive power would probably be worse when the features are less independent.
- DT: The decision tree algorithm seems to be good at predicting the classes when the feature space is low-dimensional and the features of the classes are not very similar. However as seen above, the decision tree algorithm didn’t perform well with a higher dimensional feature space.
- Bo: Based on our result, the boosted variants seem to have higher predictive power compared to the non-boosted ones (probably because it is a combination of many classifiers).

Mixed types of data

- NB: Can handle both continuous and discrete/categorical features.
- DT: Can handle both continuous and discrete/categorical features.
- Bo: Same as the basic classifier.

Scalability

- NB: The feature independence assumption makes it so that a higher dimension of the data doesn't have a big impact.
- DT: Doesn't seem to perform as well for higher dimensional feature spaces.
- Bo: Boosting can negatively impact the scalability since more memory and training time is needed.